

**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
DE SÃO PAULO**

Análise e Desenvolvimento de Sistemas

Fausto Biazzi de Sousa

Cético:

Desenvolvimento de software para auxílio na análise forense de imagens.

Campinas

2016



Fausto Biazzi de Sousa

Cético:

Desenvolvimento de software para auxílio na análise forense de imagens.

Trabalho de Conclusão de Curso apresentado ao Curso de Tecnologia em Análise e Desenvolvimento de Sistemas do Instituto Federal de Educação, Ciência e Tecnologia de São Paulo – Campus Campinas, como requisito para a obtenção do grau de Tecnólogo em Análise e Desenvolvimento de Sistemas.

Orientador: Prof. Dr. Tiago Carvalho.

Campinas

2016

S729c Sousa, Fausto Biazzi de
 Cético: desenvolvimento de software para auxílio na análise forense de
 imagem/Fausto Biazzi de Sousa – Campinas, 2016.
 44. : il.

 Orientador: Dr. Tiago Carvalho

 Monografia (Graduação) – Instituto Federal de São Paulo – Câmpus
 Campinas. Curso de Tecnologia em Análise e Desenvolvimento de Sistemas,
 2016.

 1. Forense. 2. Python. 3. OpenCV. 4. Interface gráfica de usuário. 5. Visão
 computacional. I. Instituto Federal de São Paulo - Câmpus Campinas. Curso de
 Tecnologia em Análise e Desenvolvimento de Sistemas. II. Título.

Fausto Biazzi de Sousa

Cético:

Desenvolvimento de software para auxílio na análise forense de imagens.

Trabalho de Conclusão de Curso apresentado ao Curso de Tecnologia em Análise e Desenvolvimento de Sistemas do Instituto Federal de Educação, Ciência e Tecnologia de São Paulo – Câmpus Campinas, como requisito para a obtenção do grau de Tecnólogo em Análise e Desenvolvimento de Sistemas.

Aprovado em: 25 de novembro de 2016.

BANCA EXAMINADORA

Rafael Muniz – IFSP - Campinas

Samuel Botter Martins - IFSP - Campinas

Tiago Carvalho - IFSP - Campinas (orientador)

AGRADECIMENTOS

Agradeço a minha família, que me apoiou incondicionalmente nessa jornada, em especial ao meu pai Joel Sétimo de Sousa (*in memoriam*), o qual, acredito, que estaria orgulhoso de meus esforços.

Aos homens e mulheres das polícias civis e técnico-científicas, bem como demais pesquisadores e investigadores forenses, ao qual esse projeto é dedicado e visa auxiliar.

Aos médicos, enfermeiros, bombeiros, fisioterapeutas e demais profissionais de segurança, resgate e saúde pois, provavelmente, sem eles não seria possível que eu conseguisse escrever essas palavras.

E por fim, aos funcionários do CTI e IFSP- Campus Campinas, aos meus professores e colegas, que me ensinaram e auxiliaram durante todo o desenvolvimento desse trabalho, em especial nos momentos de maior dificuldade.

“Os homens não odeiam a ilusão, mas as consequências deploráveis e hostis de certas espécies de ilusão. “

(Nietzsche; Friedrich).

RESUMO

O ser humano sempre teve necessidade de registrar seu cotidiano e, com o advento da fotografia digital, fomos praticamente inundados por fotos de eventos, pessoas e lugares seja através de das redes sociais ou dos demais meios de comunicação. No entanto, junto com essa praticidade e velocidade, as manipulações fotográficas também se espalharam, tornando-se um problema constante tanto para pesquisadores e peritos quanto para a população geral. Infelizmente, as ferramentas para se verificar a autenticidade de tais imagens não evoluíram em praticidade de uso sendo, no geral, relegadas profissionais forenses e pesquisadores. Este trabalho visa desenvolver um software capaz de executar métodos para realizar análises forenses de imagens se utilizando de técnicas de visão computacional de forma prática e simplificada.

Palavras chave: Forense. Python. OpenCV. Interface Gráfica de Usuário. Visão Computacional.

ABSTRACT

Humankind has always had the need to register its daily life, and with the advent of digital photography, we have been virtually flooded with pictures of events, people and places whether by means of social media or other means of communication. However, along with this practicality and speed, photographic manipulations have also spread out becoming an ongoing issue for both researchers and specialists, as well as the overall population. Unfortunately, the tools to ensure the authenticity of such images have not evolved in terms of ease of use being relegated to forensic professionals and researchers. This paper aims the development of a software able to execute methods to perform forensic analysis of images by using computer vision techniques in a practical and simple way.

Keywords: Forensic. Python. OpenCV. Graphic User Interface. Computer Vision.

Lista de Ilustrações

Figura 1 – Péter Guzli - "Tourist Guy" (2001).....	10
Figura 2 – Comparativo de foto de antes e depois da remoção de Trotsky (em destaque) e outros membros do Partido Comunista. URSS (1919).....	13
Figura 3 – (Imagem de divulgação) Lançamento de mísseis Iranianos em 2008.....	14
Figura 4 – Destaque dos elementos "clonados" da imagem anterior.....	14
Figura 5 – Detecção facial em aplicativo de um Sistema Operacional comercial.....	17
Figura 6 – Imagem recebida pelos pilotos do caça F-35.....	18
Figura 7 – Aspecto externo do capacete usado pelos pilotos da versão Britânica do F-35.....	18
Figura 8 – Saída em linha de comando do algoritmo "Illuminants".....	20
Figura 9 – Caixas de mensagem do Cético com resultado da análise pelo "Illuminants".....	20
Figura 10 – Esquema "topdown" previsto para utilização no desenvolvimento do Cético.....	22
Figura 11 – Interface gráfica versão Alfa do Cético.....	22
Figura 12 – Atual estrutura de arquivos do projeto.....	23
Figura 13 – Saída no prompt com dados da detecção realizada através de um algoritmo que utiliza OpenCV.....	25
Figura 14 – Módulo de detecção de rostos em funcionamento na interface principal do Cético.....	25
Figura 15 – Terminal Linux executando o software forense copy-move-detector.....	27
Figura 16 – Menu de funções do Cético.....	27
Figura 17 – Tela de seleção de arquivo a ser analisado.....	28
Figura 18 – Tela de configuração do detector de faces.....	28
Figura 19 – Exemplo de mensagem, de erro do Cético.....	28
Figura 20 – Composição de captura exibindo a resposta de uma análise de imagem sabidamente adulterada.....	29
Figura 21 – Captura de tela analisando a fotografia "situation room" por Pete Souza (2011)....	30
Figura 22 – Comparação da mesma fotografia com diferentes alvos para análise do "Illuminants".....	31

Lista de Abreviaturas e Siglas

3D	Três Dimensões
ABNT	Associação Brasileira de Normas Técnica
AFD	Análise Forense de Documentos Digitais
BSD	<i>Berkeley Software Distribution</i>
CV	<i>Computer Vision</i> (Visão Computacional)
GUI	<i>Graphic User Interface</i> (Interface Gráfica de Usuário)
IFSP	Instituto Federal de São Paulo
SVM	<i>Support Vector Machines</i> (máquina de vetores de suporte)
SO	Sistema Operacional
URSS	União das Repúblicas Socialistas Soviéticas
WTC	<i>World Trade Center</i>

SUMÁRIO

1 INTRODUÇÃO.....	10
1.1 Objetivo.....	11
1.1.1 Objetivo geral.....	11
1.1.2 Objetivos específicos.....	11
1.2 Organização do trabalho.....	11
2 JUSTIFICATIVA.....	13
3 FUNDAMENTAÇÃO TEÓRICA.....	16
3.1 A visão computacional.....	16
3.2 OpenCV.....	18
3.3 TKinter e interfaces gráficas.....	19
3.4 Método <i>Illuminants</i>	19
4 METODOLOGIA.....	21
4.1 Definição do ambiente de desenvolvimento.....	21
4.2 Definição da arquitetura de desenvolvimento.....	23
4.3 Organização e metodologia de desenvolvimento.....	24
4.4 Integração de módulos.....	24
4.4.1 Inclusão do algoritmo <i>Illuminants</i> ao Cético.....	26
4.5 Interface gráfica do usuário.....	27
5 RESULTADOS ALCANÇADOS.....	29
5.1 Dificuldades encontradas.....	30
5.1.1 Falsos positivos e negativos.....	30
5.1.2 Dificuldades com algoritmos de terceiros.....	31
5.1.3 Redistribuição.....	31
6 CONCLUSÃO.....	33
6.1 Trabalhos futuros.....	33
REFERÊNCIAS.....	34
APÊNDICE A - Instruções para execução.....	36
Anexo A – <i>_Illuminants.py</i>	37
Anexo B – Função <i>illuminants</i> em <i>funcoes.py</i>	40
Anexo C – Chamadas de aviso e erro do <i>Illuminants</i> no script “ <i>__error.py</i> ”.....	41
Anexo D – Inserção de item no menu e funções na interface gráfica.....	42

1 INTRODUÇÃO

Tanto as câmeras fotográficas quanto a própria fotografia passaram por grandes mudanças desde que foram criadas no início do século XIX (CANABARRO, 2005). As manipulações fotográficas acompanharam esta evolução e, independente das motivações e das técnicas empregadas, não é raro que tais manipulações acabem por gerar transtornos a instituições e pessoas, levando-as a acreditarem em histórias inverídicas ou propagarem padrões e conceitos por vezes absurdos. Um exemplo é o caso do “*Tourist Guy*”, cuja principal imagem pode ser conferida na Figura 1, que além de propagar uma história inverídica (na qual uma suposta foto de um turista teria sido encontrada nos escombros do *World Trade Center* (WTC) mostrando o avião que se chocou a primeira torre instantes antes do impacto), esta manipulação gerou um caso de falsidade ideológica (perpetrado por um brasileiro que se passou pelo ucraniano Péter Guzli visando fama) e fraude conforme reporta o artigo “Um turista no topo do WTC” (LOPES, 2002).

Figura 1 – Péter Guzli - “Tourist Guy” (2001)



Fonte: LOPES, 2002

Com as constantes melhorias em *hardware*, *software* e em técnicas utilizadas para edição de imagens, desmentir tais registros vêm se tornando uma tarefa cada vez mais difícil. Enquanto temos aplicativos complexos direcionados à realização de edições em imagens, o mesmo tipo de praticidade não se repete entre os profissionais forenses e demais pessoas que precisam examinar a veracidade de tais imagens. Tais indivíduos, em geral, necessitam realizar cada análise de forma independente por meio de métodos complexos, os quais muitas

vezes tornam o processo de perícia de casos de manipulação de imagem morosos e até mesmo financeiramente custosos.

No decorrer das próximas páginas, apresentaremos o desenvolvimento de um software para detecção de manipulações de fotos digitais, com capacidade de agregar módulos independentes desenvolvidos por pesquisadores da área. Esta ferramenta pode se tornar extremamente útil na dissolução de boatos e até mesmo na detecção ou resolução de crimes diversos como, por exemplo, os chamados “crimes contra a honra”, nos quais atualmente um grande número de pessoas, tanto públicas quanto anônimas, são expostas, através de manipulações fotográficas, a cenas que as colocam em situação vexatória ou considerada “socialmente reprovável”.

A ferramenta criada recebeu o nome de Cético, cuja etimologia remete ao grego ‘skeptikós’, que significa 'aquele que observa/que reflete', referenciando a ideia de que o software observará a imagem posta em dúvida procurando por evidências de manipulação.

1.1 Objetivo

1.1.1 Objetivo geral

Desenvolver um software multiplataforma, com interface gráfica, de fácil utilização, capaz de incorporar algoritmos de análise forenses já existentes na literatura, visando a análise de manipulações e fraudes em imagens digitais.

1.1.2 Objetivos específicos

- a) Desenvolvimento de uma interface gráfica agradável e de fácil utilização para público profissional e leigo;
- b) Portar a ferramenta para múltiplos ambientes operacionais, sendo capaz de ser utilizado por um público amplo sem necessidade de alteração do seu ambiente de trabalho habitual;
- c) Realizar o estudo e implementação de métodos da literatura de Análise Forense de Documentos Digitais (AFD) ao aplicativo.

1.2 Organização do trabalho

O restante deste trabalho encontra-se subdividido em: Seção 2 “Justificativa”, em que é abordada a história da manipulação fotográfica, sua evolução e as dificuldades dos

profissionais em detectar as mesmas; Seção 3 "Fundamentação teórica", em que são expostas as documentações e trabalhos similares e que servem de base para essa documentação; Seção 4 "Metodologia", que visa expor aspectos técnicos do desenvolvimento da aplicação bem como sua estrutura e tecnologias empregadas; seguido pelas Seção 5 "Resultados Alcançados" e Seção 6 "Conclusões" a respeito do desenvolvimento e por fim a seção "Referências", em que estão relacionadas às obras que serviram de base teórica e prática para este projeto.

2 JUSTIFICATIVA

O ser humano tenta registrar o mundo que o rodeia desde a antiguidade. Este desejo está gravado na rocha das cavernas, nas paredes de templos e edificações, no desenvolvimento da escrita, das artes plásticas. A partir de 1829 (LUCARINI, 2014), após o aprimoramento de técnicas que derivavam do desenvolvimento da câmara escura e de evoluções do estudo da óptica aliado à química moderna, fomos apresentados à técnica conhecida como fotografia. A palavra fotografia corresponde a junção das palavras gregas *φῶς* (*phos* do qual deriva o termo *phōtós* que em português significa “luz”) e *γραφή* (que *graphê* que pode ser traduzido como grafado/gravado/escrito/desenhado) significando, em uma interpretação livre, “desenhado com luz”.

Diferente do que ocorria em praticamente todas as técnicas de retratação que a precederam, as técnicas fotográficas possuem características que independem do humor, aspirações ou influências do profissional que as aplica. Sendo uma técnica que se baseia na captação da luz refletida no ambiente por um filme ou papel fotossensível, a ideia básica seria que todos os elementos que fossem captados pelo dispositivo fotográfico iriam contribuir pra compor uma imagem fiel do que fora inicialmente retratado. Entretanto, isso não é algo factual, pois, se utilizando de truques de iluminação, sobreposições, entre outro, fotógrafos profissionais tornaram-se capazes de manipular os resultados de suas obras fazendo com que fossem adicionados ou retirados elementos de cena conforme sua vontade ou necessidade. Um exemplo de manipulações como a descrita acima pode ser vista na Figura 2.

Figura 2 – Comparativo de foto de antes e depois da remoção de Trotsky (em destaque) e outros membros do Partido Comunista. URSS (1919)



Fonte: Elaborado pelo autor.

Devido à qualidade dos materiais utilizados, tanto a geração das fotografias e alterações, quanto a detecção de tais modificações eram igualmente difíceis. Mas isso mudou com o advento das câmeras digitais e da evolução e popularização de softwares de

processamento de imagens, tais como Adobe Photoshop®¹, cuja influência e popularidade na atual cultura fez com que o termo “photoshopar” se tornasse sinônimo de modificação de fotografias, como as representadas nas Figuras 3 e 4.

Figura 3 – (Imagem de divulgação)
Lançamento de mísseis Iranianos em 2008



Fonte: NIZZA; J. LYONS (2008)

Figura 4 – Destaque dos elementos "clonados" da
imagem anterior



Fonte: NIZZA; J. LYONS (2008)

Tendo em vista que vivemos em um mundo no qual boa parte das informações que recebemos é transmitida por meio de imagens e outros estímulos visuais, saber se o que está sendo visualizado é de fato real vem se tornando cada vez mais complicado. Isto torna necessário que as técnicas e ferramentas capazes de detectar a presença de adulterações em imagens se tornem igualmente acessíveis a jornais, instituições de pesquisa, peritos forenses e, em especial, ao cidadão comum. Para tal, este trabalho apresenta a proposta do desenvolvimento de um programa chamado Cético, o qual será voltada para detecção de alterações em imagens digitais.

A utilidade do trabalho proposto dá-se ao fato de que, frequentemente, são divulgadas fotografias digitais que podem, ou não, terem passado por um processo de adulteração. Muitas destas possíveis adulterações, por vezes, colocam anônimos e ilustres membros da sociedade nas mais diversas situações, sendo as mais comuns aquelas que implicam em situações de exposição dadas como vexatórias, em especial aquelas relacionadas à nudez ou vinculação do indivíduo retratado a uma situação social/moralmente reprovável ou, até mesmo, criminosa. Tais situações podem gerar toda uma sorte de problemas de ordem práticas, tais como, perdas financeiras, danos morais, perdas de credibilidade, dentre outras (MORAES JUNIOR, 2007).

Tendo essas situações em vista, o software Cético destina-se a oferecer uma ferramenta que seja capaz de realizar diversos tipos de análises, incorporando métodos forenses já existentes ou que venham a ser desenvolvidos, provendo ao usuário relatórios com

¹ Adobe Photoshop: <http://www.adobe.com/br/products/photoshop.html>

as saídas tratadas das análises e suas estimativas a respeito da originalidade da foto. Assim, ele serve como uma ferramenta de apoio ao usuário na formulação de uma avaliação da imagem.

3 FUNDAMENTAÇÃO TEÓRICA

Este trabalho tem como base as necessidades apresentadas tanto por profissionais como Gilmar Henrique Lopes, pesquisador e fundador do site E-Farsas², quanto por instituições como as polícias judiciárias, nas figuras das polícias Federal, Civil e suas superintendências Técnico-Científicas. Há também a necessidade de diversos profissionais, que realizam trabalho investigativo e pericial em relação a fotografias digitais, que, entre outros, motivou a criação de ferramentas como o *NuDetective* (CASTRO; ELEUTERIO, 2010), o qual foi, desenvolvido para realizar a detecção automática de nudez e auxiliar no combate a pornografia Infantil. Esta necessidade também motivou os esforços do orientador deste trabalho, o Prof. Dr. Tiago Carvalho, que descreveu métodos de análise de fraudes em fotografias empregando o mapeamento de inconsistências de iluminação de diversos elementos da fotografia no artigo *Illuminant-based Transformed Spaces for Image Forensics* (CARVALHO, et al., 2016) e *Illumination Inconsistency Sleuthing for Exposing Fauxtography and Uncovering Composition Telltales in Digital Images* (CARVALHO, 2016).

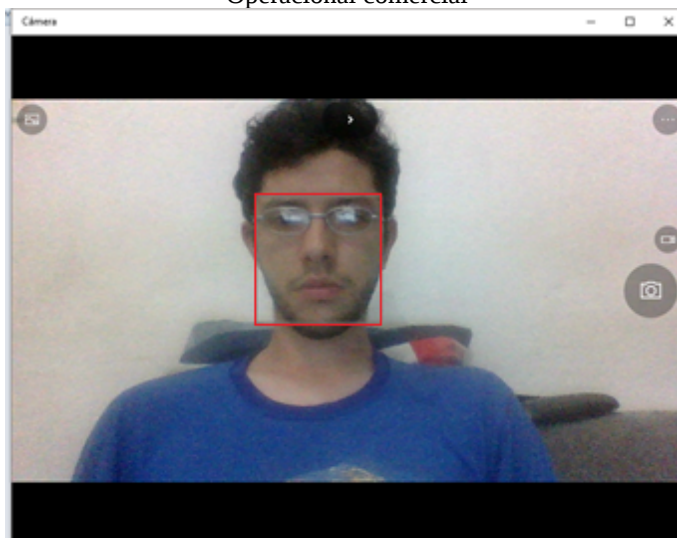
Estes trabalhos encontram-se em algoritmos de código aberto, estando disponíveis em repositórios e documentações de teses, podendo ser utilizadas por terceiros. O Cético tem como um dos objetivos propostos a capacidade de agregar módulos ou aplicativos de terceiros integrando-os a sua interface, fornecendo assim um ambiente único capaz de realizar uma grande gama de testes e emitir relatórios com os resultados. A necessidade de uma ferramenta prática e de capacidade modularização para detecção de imagens forjadas é aventada no trabalho “*A Forensic Tool for Investigating Image Forgeries*” (FONTANI, BIANCHI, et al., 2013). Sendo assim, é imprescindível o conhecimento, análise e a compreensão de tais ferramentas e métodos a fim de agregá-los ao projeto proposto.

3.1 A visão computacional

A visão computacional é a área da ciência da computação que se dedica a pesquisar e desenvolver métodos que façam com que dados, em especial, imagens digitalizadas por intermédio de câmeras, sensores e diversos outros dispositivos, possam ser utilizados por sistemas computadorizados. Estes dados poderão ser manipulados e interpretados por algoritmos para diversos propósitos tais como detecção e identificação de objetos ou pessoas, aferição de velocidade e interação, e uma infinidade de outras utilidades. Um exemplo de um aplicação de visão computacional para detecção de faces pode ser visto na Figura 5.

²<http://www.efarsas.com.br>

Figura 5 – Detecção facial em aplicativo de um Sistema Operacional comercial



Fonte: Captura de tela aplicativo de câmera do Windows 10

Este campo de estudo ainda é considerado pouco maduro uma vez que apenas recentemente os sistemas computacionais adquiriram capacidade de processamento satisfatória para lidar com volumes de dados tão grandes quanto os gerados por sistemas de análise de imagens. No entanto, suas bases remetem às décadas de 60 e 70 em trabalhos como o artigo “Olhos e Ouvidos para Computadores” (REDDY, 1973), no qual apresenta-se uma visão unificada das pesquisas relacionadas à percepção de fala e visão das máquinas até aquele momento.

É importante frisar que não existe uma divisão clara entre o que é a visão computacional e o que é processamento de imagens. Sendo assim, conceitos de ambas as áreas são muitas vezes confundidos ou mesclados. Porém, cruamente, podemos apontar que visão computacional é o conjunto de técnicas utilizadas a fim de construir a interpretação de uma imagem, a exemplo novamente da identificação da presença de rostos em uma fotografia.

Devido à capacidade de reconhecimento facial, reconhecimento de padrões de interação e objetos, entre outros fatores, os avanços nas pesquisas na área de visão computacional trazem benefícios diretos a diversas áreas tais como: climatologia, análise de fotos de satélites, medicina, entretenimento, segurança e defesa. Este último é um dos ramos com maior volume de investimento no desenvolvimento de sistemas e equipamentos que se utilizam dessa tecnologia, a exemplo do capacete desenvolvido para pilotos do caça F-35, exposto nas Figuras 6 e 7, que foi projetado empregando a visão computacional e outras tecnologias, para ter capacidade de prover realidade aumentada, imagens com dados complexos dos alvos, além da telemetria da aeronave diretamente na viseira. na viseira.

Produtos como este, chegam a custar US\$ 400.000,00 (quatrocentos mil dólares) a unidade (SWARTS, 2015).

Figura 6 – Imagem recebida pelos pilotos do caça F-35.



Fonte: SWARTS (2015)

Figura 7 – Aspecto externo do capacete usado pelos pilotos da versão Britânica do F-35.



Fonte: SWARTS (2015)

Infelizmente tal volume de investimentos e principalmente os benefícios de tais tecnologias de visão computacional, mesmo que em estado embrionário (como é o caso dos sistemas desenvolvidos para o supracitado F-35), estão longe de muitas pessoas e órgãos que deles necessitam. Situação esta que o desenvolvimento das bibliotecas e ferramentas *Open Source* tais como a OpenCV buscam amenizar.

3.2 OpenCV

O OpenCV é uma biblioteca de código aberto voltada para visão computacional e aprendizado de máquina (*machine learning*), com seu uso regularizado sob a licença *Berkeley Software Distribution (BSD)*. Foi construído para fornecer uma infraestrutura para aplicações que se utilizam de visão computacional, de modo a facilitar seu uso em produtos comerciais. A biblioteca tem mais de 2500 algoritmos otimizados que podem ser utilizados para detectar e reconhecer rostos, identificar objetos, classificar as ações humanas em vídeos, extrair modelos 3D de objetos, encontrar imagens similares em um banco de imagens, remover olhos vermelhos de imagens tiradas com flash, seguir os movimentos dos olhos, reconhecer cenários e estabelecer marcadores para sobrepor-los com técnicas de realidade aumentada, entre outros. Segundo seus mantenedores³, a OpenCV possui uma grande comunidade ativa e número estimado de download superior a 7 milhões. É amplamente utilizada por empresas,

³<http://opencv.org/about.html>

grupos de pesquisa e organismos governamentais em projetos dos mais variados níveis de complexidade e aliados a algoritmos de reconhecimento de padrões, como o *Support Vector Machines* (LORENA; CARVALHO, 2007), potencialmente podendo ser utilizada para criação de qualquer ferramenta de análise ou detecção envolvendo imagens.

3.3 TKinter e interfaces gráficas

TKinter é uma das bibliotecas disponíveis para a linguagem de programação Python3 criada para permitir o desenvolvimento de interfaces gráficas e faz parte do pacote básico do Python em algumas distribuições (LABAKI, 2005). O TKinter pode ser executado em ambientes operacionais diversos sem grandes problemas, tendo sido testado durante o desenvolvimento deste projeto tanto em computadores executando os sistemas operacionais Windows 7 e 10, quanto Linux (distribuições diversas). Esta capacidade é atraente uma vez que um dos objetivos deste trabalho é fazer com que a ferramenta desenvolvida atinja o maior número e variedade de usuários possível. Esta intenção também justifica a interface gráfica no que tange a familiaridade por parte de usuários com menor intimidade com computadores, uma vez que uma expressiva parcela dos algoritmos forenses disponíveis atualmente tem sua utilização através de linhas de comando, o que pode inibir usuários mais leigos.

3.4 Método *Illuminants*

O método *Illuminants*, o qual foi desenvolvido pelo orientador deste trabalho, consiste na análise das inconsistências de iluminação de uma imagem por meio de um processo de segmentação e extração de vetores com base em até 54 descritores de imagem que são escolhidos mediante parâmetros passados pelo usuário. Sendo, atualmente, seu ponto focal a comparação das características de iluminação de rostos.

Os vetores extraídos por cada descritor de imagem são então encaminhados para uma SVM que acompanha o código, esta SVM é treinada para analisar tais dados e realizar votações a respeito da autenticidade da imagem. Esta votação tem como resultado possíveis as saídas “Fake” ou “Normal”. Caso ocorra empate entre as votações dos descritores na votação final ou a imagem não consiga ter seus descritores selecionados devidamente extraídos o algoritmo assume a imagem como “Fake”.

Os resultados de uma análise de imagem pode ser vista na Figura 8.

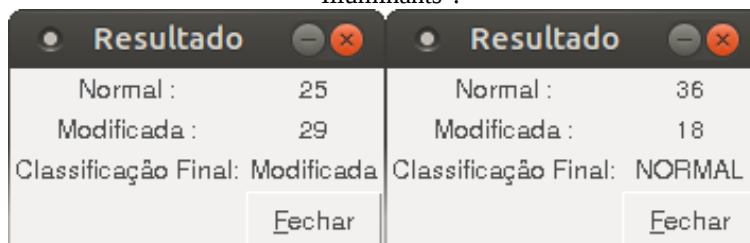
Figura 8 – Saída em linha de comando do algoritmo "Illuminants"

```
( 'sasi', 0, 0, 'IIC' )
( 'sasi', 2, 2, 'IIC' )
( 'sasi', 4, 2, 'GGE' )
( 'sasi', 0, 0, 'GGE' )
( 'sasi', 2, 2, 'GGE' )
( 'spytec', 4, 2, 'IIC' )
( 'spytec', 0, 0, 'IIC' )
( 'spytec', 2, 2, 'IIC' )
( 'spytec', 4, 2, 'GGE' )
( 'spytec', 0, 0, 'GGE' )
( 'spytec', 2, 2, 'GGE' )
( 'unser', 4, 2, 'IIC' )
( 'unser', 0, 0, 'IIC' )
( 'unser', 2, 2, 'IIC' )
( 'unser', 4, 2, 'GGE' )
( 'unser', 0, 0, 'GGE' )
( 'unser', 2, 2, 'GGE' )
Votos Normal: 36
Votos Fake: 18
Classificacao Final: NORMAL
Concluida analise SVM
```

Fonte: Captura de tela em ambiente Linux.

Durante a adaptação tais resposta do algoritmo foram simplificadas e convertidas para termos mais formais conforme pode ser visto na Figura 9.

Figura 9 – Caixas de mensagem do Cético com resultado da análise pelo "Illuminants".



Fonte: Elaborado pelo autor.

4 METODOLOGIA

O desenvolvimento conta com o auxílio de especialistas na área de desenvolvimento forense, uma vez que o Cético tem como proposta tornar mais prática a análise de fotos digitais por meio da realização de testes automatizados nos arquivos selecionados. A proposta desse software é utilizar técnicas como, por exemplo, a análise de iluminação (CARVALHO, et al., 2016), busca por trechos copiados entre partes da imagem (CARVALHO, 2014), entre outras. Tais algoritmos são conhecidos na literatura e encontram-se disponíveis em ferramentas atualmente independentes tendo seus códigos fontes disponíveis nos trabalhos referenciados.

4.1 Definição do ambiente de desenvolvimento

O desenvolvimento foi realizado utilizando a linguagem de programação Python, mais especificamente a versão 3.5.1, devido a sua disponibilidade em diversas plataformas computacionais, ampliando dessa forma as instituições e o público que poderá utilizá-lo e facilitando sua disseminação junto aos profissionais e leigos que necessitem de ferramentas deste tipo sem modificar seu ambiente de trabalho habitual.

O Python3 foi escolhido em detrimento das versões 2.x, pois, conforme consta no site oficial da linguagem, a versão 3.x é “o presente e futuro da linguagem Python” (COGHLAN, 2015). As versões anteriores receberam a identificação “*legacy*” (legado), ou seja, seguem sendo distribuídas, recebendo suporte voltado para manutenção e continuidade de código já consolidado, sendo recomendado pelos mantenedores que novos códigos e o aprendizado de novos usuários ocorram utilizando-se as versões mais modernas. No entanto, é bom ter em vista que ao realizar tal opção de versão, existe a possibilidade de surgirem problemas com módulos a serem integrados. Pode existir também a incapacidade de executar os aplicativos em determinados ambientes operacionais por conta do Python 3.0 não possuir determinadas bibliotecas ou chamadas de sistema.

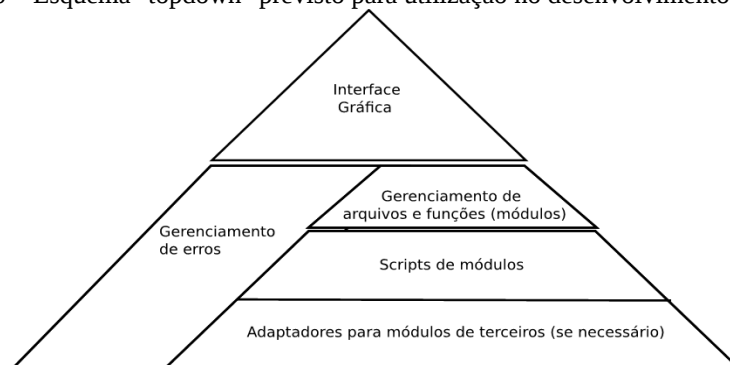
O ambiente de desenvolvimento escolhido para edição foi o Anaconda 4.0.5 devido a fato dela ser um ambiente bem integrado com a versão 3.5 do Python e fornecer diversas bibliotecas de desenvolvimento em seus repositórios, reduzindo assim o tempo de *setup* do ambiente, tendo o código sido escrito majoritariamente na IDE PyCharm.

Em um primeiro momento, o projeto foi concebido para que sua primeira fase contemplasse o estudo e adaptação dos métodos de entrada de algoritmos já existentes e disponíveis de forma a serem acionados pelo Cético sem necessidade de comandos adicionais

do operador. Um módulo de controle para importação e seleção dos processos a serem executados e, por fim, uma interface gráfica no qual o usuário irá interagir.

Essa estrutura resultaria em um sistema de desenvolvimento em camadas, separando camada de Dados (carregamento de módulos de análise e das imagens a serem analisadas), camada de negócios (gerenciamento dos algoritmos a serem utilizados pelo usuário) e camada de apresentação (interface gráfica com a qual o usuário irá interagir). Esta proposta está representada graficamente na Figura 10.

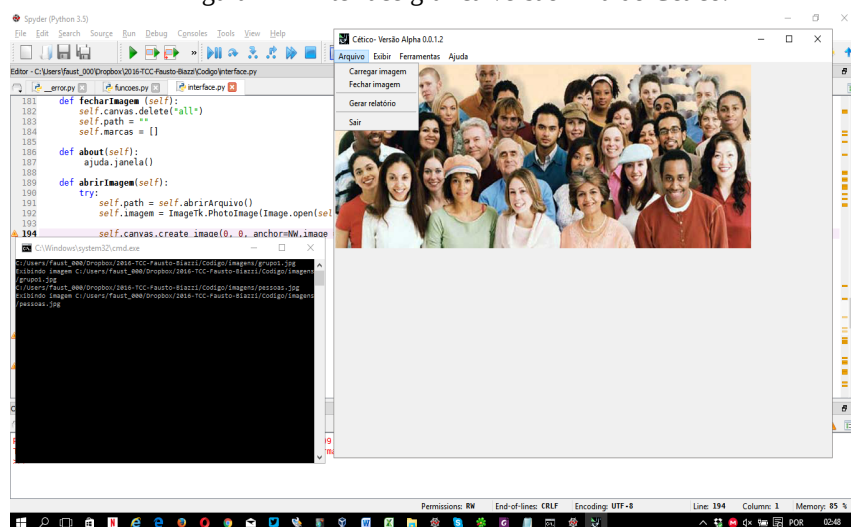
Figura 10 – Esquema "topdown" previsto para utilização no desenvolvimento do Cético



Fonte: Elaborado pelo autor.

Porém, devido à complexidade de realização de alguns procedimentos nas fases iniciais do projeto, o mesmo foi reestruturado de forma a iniciar o desenvolvimento pela criação da interface gráfica do software utilizando a biblioteca TKinter, atualmente na versão 8.6, antes da integração dos módulos de detecção e análise. Desta forma, ocorre um *feedback* visual, a exemplo da Figura 11, que permite um acompanhamento mais práticos dos resultados durante a aplicação dos módulos.

Figura 11 – Interface gráfica versão Alfa do Cético.



Fonte: Captura de tela em ambiente Windows.

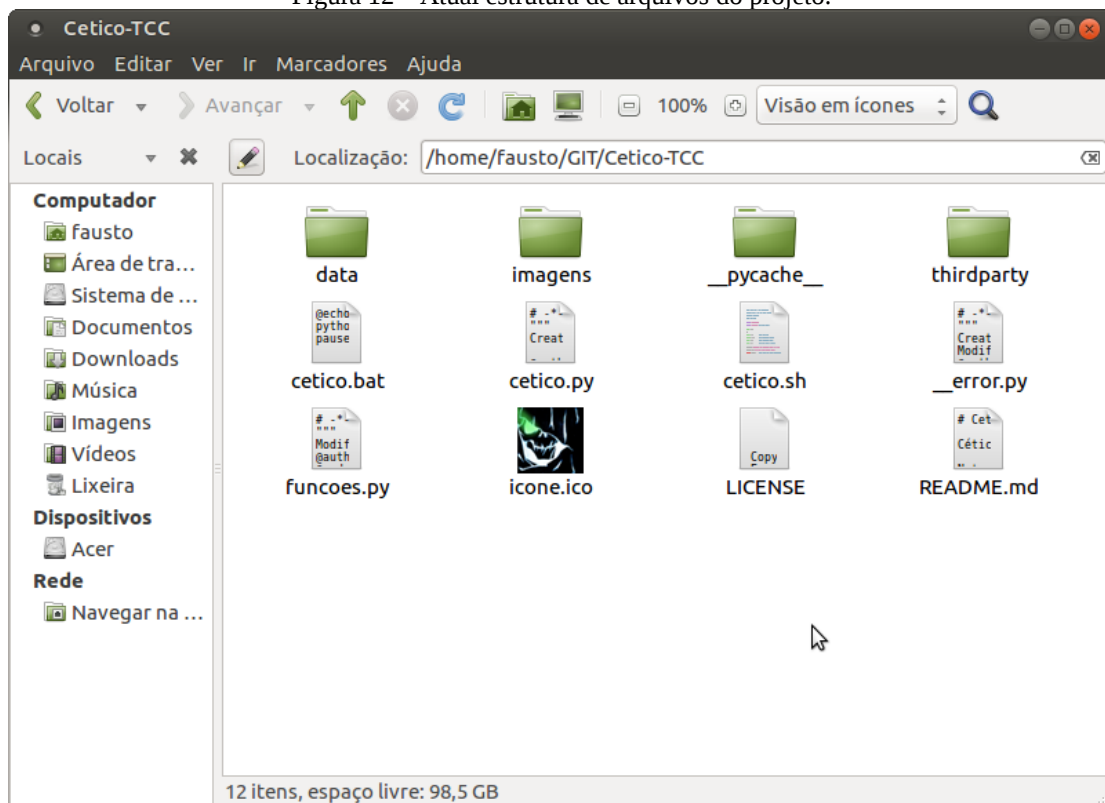
É desejável que o desenvolvimento da interface gráfica do software siga padrões que o torne familiar para os futuros usuários. Para tal, ao longo do desenvolvimento deste trabalho, recorremos a referências de design de aplicativos tanto da área de edição de imagens, a exemplo do Adobe Photoshop®, quanto de aplicativos de uso forense como o já referenciado *NuDetective*, nos quais as tarefas são facilitadas através de menus, botões, opções comuns e de fácil identificação por parte do usuário e operados por meio de cliques ou atalhos de teclado. Sendo assim, o desenvolvimento da interface gráfica foi uma prioridade ao longo do trabalho.

4.2 Definição da arquitetura de desenvolvimento

Como citado anteriormente, a arquitetura em camadas foi o modelo adotado. Isso se deu devido a tal estrutura permitir a mudança de implementação de uma camada sem afetar as demais, o que possibilita que diferentes versões de uma camada como, por exemplo, uma interface desenvolvida utilizando a biblioteca QT ao invés da TKinter tenha acesso às funções da camada de negócios sem grandes necessidades de adaptação.

Atualmente o sistema conta com a seguinte estrutura de arquivos , ilustrada na Figura 12.

Figura 12 – Atual estrutura de arquivos do projeto.



Fonte: Captura de tela em ambiente Linux.

Dentro dessa estrutura, vale destacar os seguintes itens:

- *cetico.py* – Desenvolvido utilizando a biblioteca TKinter, é o script responsável pela geração da interface gráfica do usuário e acesso as funções do programa (camada de apresentação);
- *__error.py* – Este script pertence a camada de negócios e é responsável pelo gerenciamento dos erros do sistema em todas as camadas;
- *funcoes.py* – Originalmente planejado para integrar a camada de negócios, atualmente acumula funções da camada de dados;
- *data<pasta>* – Local para armazenamento de módulos próprios, arquivos e *scripts* secundários utilizados pela interface principal; e
- *Thirddyparty <pasta>* – Local para armazenamento de módulos de terceiros e, se necessário, scripts auxiliares para que possam ser executados dentro do ambiente do Cético;

4.3 Organização e metodologia de desenvolvimento

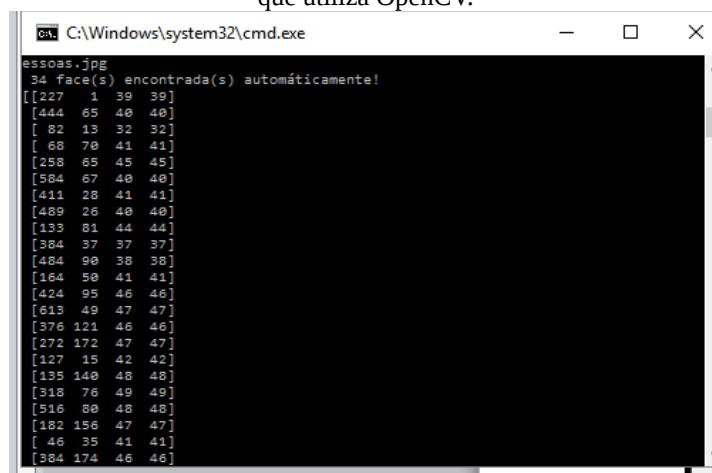
Cada um dos itens citados nas seções acima foi subdividido em tarefas menores para realização de entregas de código útil em períodos, que preferencialmente não extrapolassem 15 dias, seguindo alguns dos conceitos propostos na metodologia ágil (BECK et al., 2001). Esta metodologia de desenvolvimento foi escolhida por ser uma das mais difundidas na atualidade no mercado de trabalho, devido ao fato de permitir uma rápida percepção da evolução do código e facilitar a detecção de pequenos erros. Ela também facilita a realização de alterações em curto espaço de tempo, de forma a não comprometer o prazo final de entrega.

4.4 Integração de módulos

Um dos principais problemas a serem solucionados ao longo do desenvolvimento do projeto foi a questão da criação, adaptação e integração de módulos diversos para utilização através da interface gráfica, em especial os algoritmos desenvolvidos por terceiros. Isso ocorre devido à grande variedade de bibliotecas e mecanismos utilizados pelos mesmos, com alguns necessitando de passos complexos ou requisitos específicos para realização da sua operacionalização. De forma a iniciar os testes do sistema e das rotinas automatizadas, o primeiro módulo de visão computacional criado para a interface foi um módulo de detecção de faces da biblioteca OpenCV 3.1.0.

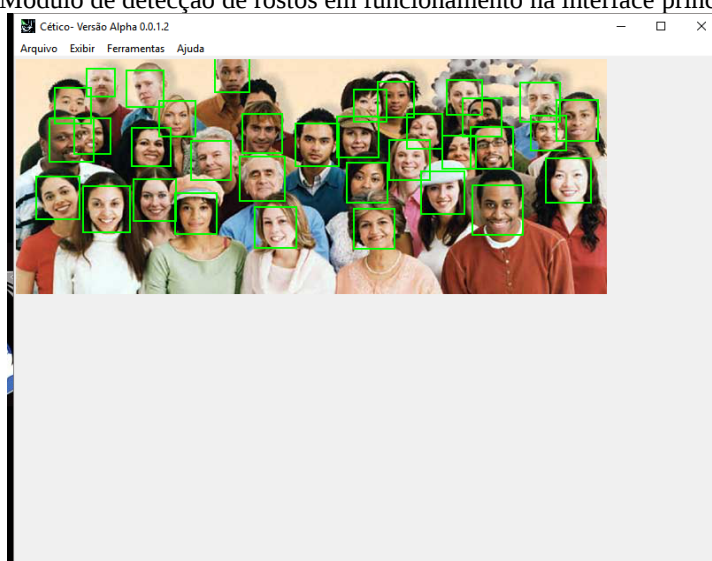
O módulo de detecção facial foi incorporado ao sistema utilizando como base o tutorial provido pelos mantenedores do OpenCV⁴, o qual propõe uso do recurso chamado *Cascade* pertencente à biblioteca OpenCV, baseado no método de detecção proposto no trabalho de Viola e Jones (VIOLA; JONES, 2001). O algoritmo pode ser usado para identificar determinados tipos de padrão com base em um arquivo XML, no caso deste projeto, faces (preferencialmente frontais). O método se mostrou bastante eficaz quando utilizado em fotos coloridas com boa resolução, com alguns poucos casos de detecção incorreta ou situações análogas, tendo seu resultado sido exposto na saída do *prompt* exposta na Figura 13 e nas marcações exibidas na Figura 14.

Figura 13 – Saída no prompt com dados da detecção realizada através de um algoritmo que utiliza OpenCV.



Fonte: Captura de tela.

Figura 14 – Módulo de detecção de rostos em funcionamento na interface principal do Cético



Fonte: Captura de tela.

⁴ http://docs.opencv.org/3.1.0/d7/d8b/tutorial_py_face_detection.html#gsc.tab=0

A incorporação deste módulo ao sistema teve como principal objetivo auxiliar o usuário na realização das marcações a serem utilizadas para alguns dos demais métodos que foram ou que possam ser incorporados ao sistema.

Outro módulo já incorporado ao software é o denominado *Illuminants*, o qual teve seu funcionamento descrito na seção 3.4 deste trabalho, este módulo tem como interface padrão entradas por linha de comando e por agregar diversos módulos e ferramentas desenvolvidas em linguagens de programação distintas necessitou de procedimentos específicos para sua inclusão a interface do Software Cético. Tais procedimento está descrito na seção a seguir, e podem servir de referência para futuras inserções de módulos desenvolvidos por terceiros.

4.4.1 Inclusão do algoritmo *Illuminants* ao Cético

Conforme é visto na seção 4.2., o software Cético é dividido em três scripts *Python* com funções distintas. Sendo assim, para realizar as alterações e inclusões de módulos à interface, deve-se alterar tais arquivo. Nos parágrafos abaixo, será exposto o processo utilizado para inclusão do módulo *Illuminants*. No entanto, é necessário informar que atualmente este algoritmo está disponível somente para ambiente Linux, e necessita ter diversos componentes compilados de acordo com as instruções contidas em seu repositório oficial. Este tipo de cenário pode não ser necessário ou possível dependendo da estrutura ou linguagem utilizada por outros algoritmos.

Uma vez realizados os procedimentos acima, o processo para inclusão do algoritmo *Illuminats* se deu por meio da da criação de um *script* nomeado “_illuminants.py” (disponível para consulta no Anexo A) que tem como função realizar tarefas assessorias tais como exclusão de arquivos temporários e ativação dos processos do algoritmo *Illuminants*, presentes na pasta “sourcecodes”, na sequência e de acordo com os paramentos passados a ele pelo *script* “funcoes.py”. O *script* “funcoes.py” por sua vez possui uma função chamada “modulo illuminant” (disponível no Anexo B), a qual recebe os parâmetros do usuário, fornecidos através da interface gráfica do Cético, e executa as funções correspondentes presentes no *script* “_illuminants.py” coletando seus retornos, para então serem repassados tanto para a interface gráfica quanto, para o *script* “__error.py” o qual retornará alertas (disponíveis no Anexo C) de acordo com a necessidade.

Por fim, a chamada para o módulo se dá através da criação de uma entrada junto ao bloco de definição do menu principal e de uma função que realize o acesso ao script

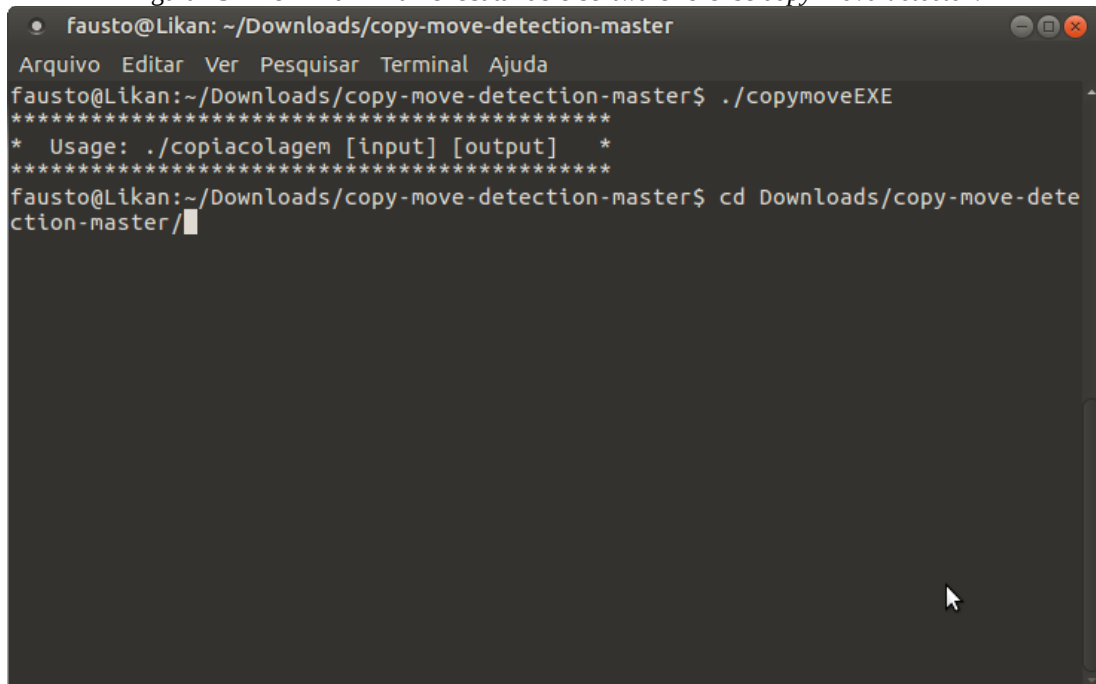
“funcoes.py”, a criação de tais entradas e funções devem ser realizadas no *script* “cético.py” de acordo com o exemplo disponível no Anexo D.

Essa estrutura visa minimizar o acúmulo de funções em um único script, facilitando a manutenção e inclusão de novos sem o risco de interferência entre funções distintas.

4.5 Interface gráfica do usuário

A interface do cético foi desenvolvida inspirada em softwares de edição de imagens tradicionais como Adobe Photoshop®, Gimp, dentre outros. Tal decisão de projeto foi tomada buscando produzir um software amigável ao usuário. Pois, a maioria dos métodos forenses da literatura, não apresentam tal facilidade para o usuário, como mostra a Figura 15.

Figura 15 – Terminal Linux executando o software forense *copy-move-detector*.

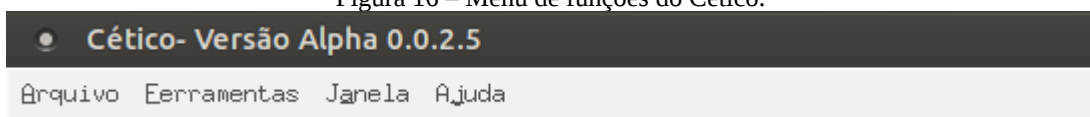


```
fausto@Likán: ~/Downloads/copy-move-detection-master
Arquivo Editar Ver Pesquisar Terminal Ajuda
fausto@Likán:~/Downloads/copy-move-detection-master$ ./copymoveEXE
*****
* Usage: ./copiacolagem [input] [output] *
*****
fausto@Likán:~/Downloads/copy-move-detection-master$ cd Downloads/copy-move-detection-master/
```

Fonte: Captura de Tela.

No cético, tal como maioria dos aplicativos para *desktop* modernos, as funções são acessadas por meio de um menu de funções conforme pode ser visto na Figura 16.

Figura 16 – Menu de funções do Cético.

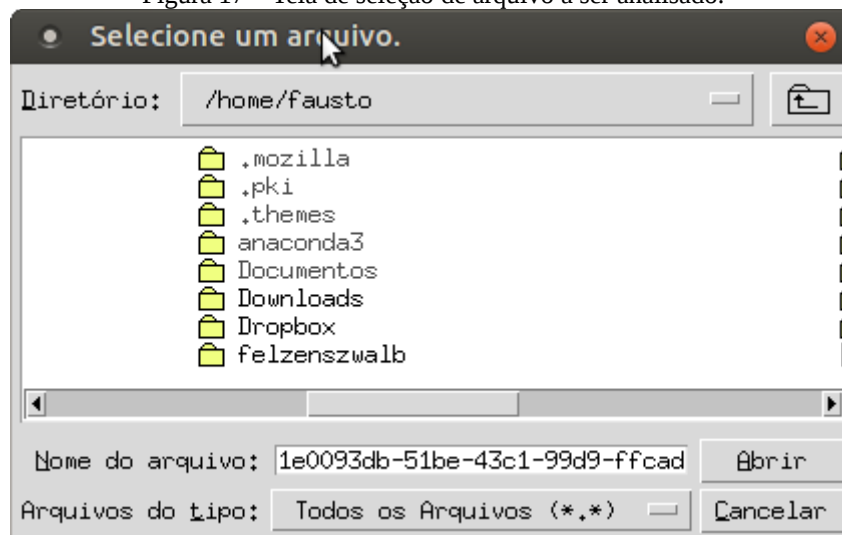


Fonte: Captura de Tela.

Por meio do menu é possível abrir imagens a serem analisadas utilizando métodos de busca de arquivos familiares como o exposto na Figura 17, executar funções como habilitar e

desabilitar a marcação manual, habilitar a detecção automática de faces, desfazer marcações e executar os módulos disponíveis de forma prática.

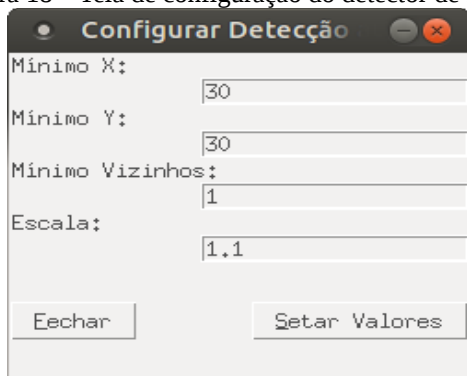
Figura 17 – Tela de seleção de arquivo a ser analisado.



Fonte: Captura de Tela.

A utilização de uma interface gráfica também permite a configurações de funções a exemplo da Figura 18 e a comunicação com o usuário indicando problemas ou fornecendo instruções de forma inteligível, conforme pode ser visto na Figura 19.

Figura 18 – Tela de configuração do detector de faces.



Fonte: Captura de tela.

Figura 19 – Exemplo de mensagem, de erro do Cético.



Fonte: Captura de tela.

5 RESULTADOS ALCANÇADOS

Após a definição dos módulos a serem implementados e efetivo desenvolvimento da aplicação foi necessária a realização da avaliação do funcionamento do código. Para isso, foram realizadas buscas na internet com o intuito de localizar imagens para serem analisadas pelo aplicativo.

Na Figura 20 temos uma composição de capturas de tela da atual versão do Cético, durante a execução de um teste utilizando a detecção de faces e o método *Illuminants*.

Figura 20 – Composição de captura exibindo a resposta de uma análise de imagem sabidamente adulterada.



Fonte: Elaborado pelo autor.

É possível perceber na figura supracitada que o algoritmo testado respondeu corretamente a respeito da autenticidade da imagem retirada do artigo “Rei do Photoshop e suas montagens com celebridades” (SILVA, 2015) tendo um total de 25 descritores analisados pelo mecanismo SVM apontando para características de iluminação consideradas dentro do padrão “Normal” contra 29 descritores apontando para a possibilidade da imagem ser “Modificada”.

As análises produzidas pelo Cético também se mostraram consistentes em casos de fotos sabidamente verdadeiras, desde que as condições de iluminação e de qualidade da fotografia se mantenham adequadas, a exemplo do exposto na Figura 21.

Figura 21 – Captura de tela analisando a fotografia "situation room" por Pete Souza (2011).



Fonte: Elaborado pelo autor.

A imagem referenciada acima, por sua vez, teve um resultado condizente com o esperado tendo como base os pontos selecionados para análise, em vermelho, tal resultado foi um índice de 31 dos descritores analisados retornando condições normais de iluminação contra 23 que detectaram alguma anormalidade, sendo a imagem por fim considerada normal.

5.1 Dificuldades encontradas

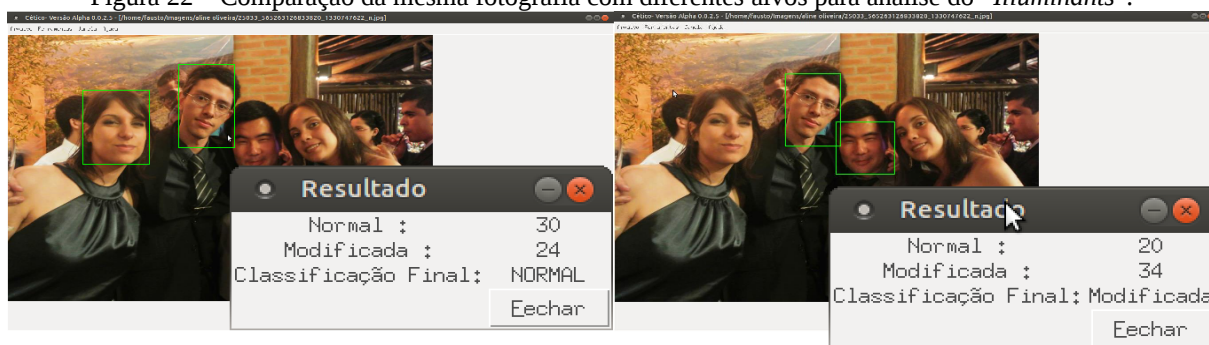
Infelizmente junto aos resultados favoráveis citados acima, também foram registrados situações de problemas durante o desenvolvimento e testes, sendo as mais problemáticas:

5.1.1 Falsos positivos e negativos

Consiste na resposta incorreta a uma situação previamente conhecida, como por exemplo, a detecção de um terceiro rosto em uma imagem com somente dois indivíduos, por parte de um algoritmo de busca por face. Tanto no uso do método *Illuminants*, quanto na utilização o módulo de detecção automática de faces, tais problemas foram encontrados.

Em geral tais resultados são causados por problemas envolvendo baixa qualidade da fotografia, utilização de filtros, e fotografias em locais ou situações com grande variação de iluminação, conforme exemplificado na Figura 22.

Figura 22 – Comparação da mesma fotografia com diferentes alvos para análise do “*Illuminants*”.



Fonte: Arquivo pessoal do autor.

Na situação mencionado acima a mesma imagem, cuja a autenticidade é de conhecimento do autor, foi avaliada alterando-se somente um dos “alvos” (marcações em verde) tendo seu resultado alterado significativamente. Tal situação, nesse caso, deu-se pela drástica alteração de iluminação entre as faces selecionadas decorrentes da iluminação utilizada no local aliada a posição dos indivíduos fotografados, que favorecem tais alterações.

Tais problemas podem ser amenizados, e até mesmo sanados, através do aprimoramento dos algoritmos e dos modelos utilizados, não sendo um problema relacionados diretamente a interface gráfica ou adequações realizados no decorrer do projeto

5.1.2 Dificuldades com algoritmos de terceiros

Uma expressão que marcou todo o desenvolvimento deste projeto foi “morrer na academia”, se referindo a projetos promissores que aparentemente só foram desenvolvidos visando a obtenção da graduação ou titulação e após isso abandonados. Tal constatação se deu quando outros algoritmos de análises forenses, além dos já incorporados ao projeto, foram estudados para inclusão no Cético. Ainda nessa primeira fase de desenvolvimento, no entanto, dificuldades para contato com os desenvolvedores, ausência de documentação de licença e de apoio, inexistência de repositórios ou códigos fontes incompletos e, muitas vezes, a não manutenção dos códigos, aliada a descontinuidade de bibliotecas utilizadas impossibilitavam completamente a utilização desses trabalhos.

5.1.3 Redistribuição

Muito embora tenha-se atingido grande parte dos intentos no que se diz facilitar a utilização de métodos forenses de análise de imagens com poucos cliques e treinamento simplificado, o Cético ainda não é uma ferramenta de fácil distribuição, o que ainda pode

afastar grande parte do público com relação a sua adoção e utilização, em especial devido as particularidades de cada módulo de análise forense que venha a ser integrado ao sistema.

Essa dificuldade de distribuição se mostra na necessidade de realização de diversos passos para a simples execução da interface principal, detalhados no Apêndice A desta documentação, e a atual impossibilidade de criação de um processo prático para download de dependências e compilação dos módulos de terceiros, conforme já foi explicado na seção 4.4.1 deste trabalho.

6 CONCLUSÃO

Nesta fase de desenvolvimento, mesmo com os problemas detectados e comentados nas seções anteriores, é seguro dizer que o software Cético encontra-se estável e bem integrado aos módulos para de detecção de faces e *Illuminants*, sendo apto ao recebimento de novos algoritmos para a incrementação de seus recursos. mostrando que:

a) O desenvolvimento de uma ferramenta prática, multiplataforma com capacidade de inserção de novos módulos é possível.

b) Se seguir em desenvolvimento constante, o Cético possui potencial para se tornar uma ferramenta completa para análises forenses.

6.1 Trabalhos futuros

Na intenção de sanar as dificuldades de distribuição, e permitir uma maior gama de algoritmos com o mínimo de transtornos para o usuário, independente da máquina ou sistema operacional utilizado, as próximas etapas de desenvolvimento do Cético se focarão em portá-lo para web através de bibliotecas como Django ou similares.

REFERÊNCIAS

- BECK, K. et al. **Princípios por trás do manifesto ágil**. Disponível em: <<http://www.agilemanifesto.org/iso/ptbr/principles.html>>. Acesso em: 15 mar. 2016.
- CANABARRO, IVO, **Fotografia, história e cultura fotográfica: aproximações** Estudos Ibero-Americanos. 2005. Disponível em: <<http://www.redalyc.org/articulo.oa?id=134618596003>>. Acesso em: 15 mar. 2016 ISSN 0101-4064
- CARVALHO, Tiago. **Illumination Inconsistency Sleuthing for Exposing Fauxtography and Uncovering Composition Telltales in Digital Images**. 2014. 153 f. Tese (Doutorado) - Curso de Ciência da Computação, Instituto de Computação, Universidade Estadual de Campinas, Campinas, 2014.
- CARVALHO, Tiago et al. Illuminant-based Transformed Spaces for Image Forensics. **Ieee Trans.inform.forensic Secur.**,[s.l.], p.1-1, 2016. Institute of Electrical & Electronics Engineers (IEEE). <http://dx.doi.org/10.1109/tifs.2015.2506548>.
- COGHLAN, Nick. Python Org. **Python2 or Python3: Should I use Python 2 or Python 3 for my development activity?**. 2015. Disponível em: <<https://wiki.python.org/moin/Python2orPython3>>. Acesso em: 14 abr. 2016.
- DOWNEY, Allen B.. **Think Python: How to Think Like a Computer Scientist**. North, Sebastopol, Ca: O'reilly Media, 2012. 298 p. ISBN: 978-1-449-33072-9.
- FARID, Hany. Perícia de imagens digitais. **Scientific American Brasil**, [s. L.]. Disponível em: <http://www2.uol.com.br/sciam/reportagens/pericia_de_imagens_digitais.html>. Acesso em: 03 maio 2016.
- FONTANI, Marco et al. A Forensic Tool for Investigating Image Forgeries. **International Journal Of Digital Crime And Forensics**, [s.l.], v. 5, n. 4, p.15-33, 2013. IGI Global. <http://dx.doi.org/10.4018/ijdcf.2013100102>.
- LABAKI, Josué. **Introdução a - Python Módulo C: TKinter**. 2005. Disponível em: <<http://www.dcc.ufrj.br/~fabiom/mab225/tutorialtkinter.pdf>>. Acesso em: 05 abr. 2016.
- LOPES, Gilmar Henrique. **Um turista no topo do World Trade Center**. 2002. Disponível em: <<http://www.e-farsas.com/um-turista-no-topo-do-world-trade-center.html>>. Acesso em: 02 mar. 2016.
- LORENA, Ana Carolina; CARVALHO, André C. P. L. F. de. Uma Introdução às Support Vector Machines. **Rita: Revista de Informática Teórica e Aplicada**, [porto Alegre,rs], v. , n. 2, p.43-67, fev. 2007. Semestral. Disponível em: <http://www.seer.ufrgs.br/rita/article/viewArticle/rita_v14_n2_p43-67>. Acesso em: 27 out. 2016.
- LUCARINI, Mariana. **História da fotografia**. 2014. Disponível em: <<http://www.alfabetizacaovisual.com.br/wp-content/uploads/2014/10/história-da-fotografia.pdf>>. Acesso em: 14 mar. 2016.

MORAIS JÚNIOR, Ívano José Genuino de. Os crimes contra a honra e a indústria do dano moral: Uma relação cada vez mais acentuada. **Âmbito Jurídico**, Rio Grande, v. X, n.44, ago. 2007. Disponível em: <http://www.ambito-juridico.com.br/site/index.php?n_link=revista_artigos_leitura&artigo_id=4336>. Acesso em: 15 mar. 2016.

NIZZA, M.; J. LYONS, P. **In an Iranian Image, a Missile Too Many**. 2008. News. Disponível em: <http://thelede.blogs.nytimes.com/2008/07/10/in-an-iranian-image-a-missile-too-many/?_r=2>. Acesso em: 20 mar. 2016.

POLASTRO, Mateus De_castro; ELEUTERIO, Pedro Monteiro da Silva. NuDetective: A Forensic Tool to Help Combat Child Pornography through Automatic Nudity Detection. **2010 Workshops On Database And Expert Systems Applications**, [s.l.], p.349-353, ago. 2010. Institute of Electrical & Electronics Engineers (IEEE). <http://dx.doi.org/10.1109/dexa.2010.74>.

REDDY, Raj. Eyes and ears for computers. In: CONFERENCE ON COGNITIVE PROCESSES AND ARTIFICIAL INTELLIGENCE, 1973, Hamburg. **Eyes and ears for computers**. Pensilvânia: Carnegie Mellon University - Computer Science Department, 1973. Disponível em: <<http://repository.cmu.edu/cgi/viewcontent.cgi?article=3009&context=compsci>>. Acesso em: 01 nov. 2015.

SILVA, E. ; CARVALHO, T. ; FERREIRA, A. ; ROCHA, A. . Going deeper into copy-move forgery detection: exploring image telltales via multi-scale analysis and voting processes. *Journal of Visual Communication and Image Representation (Print)*, v. 99, p. 1-17, 2015”.

SILVA, Christofer C. (Comp.). **Rei do photoshop e suas montagens com celebridades**. 2015. Disponível em: <<http://www.pitacodoblogueiro.com.br/rei-do-photoshop-e-suas-montagens-com-celebridades/>>. Acesso em: 10 out. 2016.

STRINGHINI, Denise; MARENGONI, Maurício. **Tutorial: Introdução à Visão Computacional usando OpenCV**. Rita: Revista de Informática Teórica e Aplicada, [porto Alegre,RS], v.16 , n. 1, p.125-160, jul. 2009. Semestral. ISSN 0103-4308. Disponível em: <http://seer.ufrgs.br/rita/article/viewFile/rita_v16_n1_p125/7289>. Acesso em: 24 set. 2015.

SWARTS, Phillip. **F-35 helmet costs \$400,000: 4 times that of predecessor**. 2015. Disponível em: <<http://www.airforcetimes.com/story/military/2015/10/26/f-35-helmet-costs-400000-4-times-predecessor/73826180/>>. Acesso em: 18 dez. 2015.

TURINO, Thiago de Castro; SOARES, Simon Medeiros. **Rastreamento de Objetos utilizando Processamento de Imagem**. 2010. 55 f. TCC (Graduação) - Curso de Engenharia Eletrônica e de Computação, Departamento de Eletrônica e de Computação - Escola Politécnica, Universidade Federal do Rio de Janeiro, Rio de Janeiro, RJ, 2010. Disponível em: <<http://monografias.poli.ufrj.br/monografias/monopoli10002881.pdf>>. Acesso em: 26 set. 2015.

VIOLA, P.; JONES, M. **Rapid Object Detection using a Boosted Cascade of Simple Features**. . In: COMPUTER VISION AND PATTERN RECOGNITION, 2001. CVPR 2001. PROCEEDINGS OF THE 2001 IEEE COMPUTER SOCIETY CONFERENCE ON. IEEE, 2001Disponível em: <<https://www.cs.cmu.edu/~efros/courses/LBMV07/Papers/viola-cvpr-01.pdf>>. Acesso em: 10 maio. 2016.

APÊNDICE A - Instruções para execução

Procedimentos para instalação do python3 e demais dependências para execução do cético.

ATENÇÃO: estes procedimentos são exclusivamente para execução da aplicação principal, os procedimentos para compilação e execução dos módulos de terceiros tais como *Illuminants* devem ser consultados em sua documentação própria.

1. - Realize o download ou clone o repositórios <https://github.com/FBSLikar/Cetico-TCC>
2. - Entre no site <https://www.continuum.io/downloads> e siga as instruções para download e instalação da plataforma Anaconda 4.2 de acordo com seu sistema operacional e arquitetura de processador.
- 3 – Acesse a linha de comando ou terminal de seu sistema operacional (windows trata-se do prompt de comando) e execute o seguinte comando:

conda create -n opencv numpy scipy scikit-learn matplotlib python=3

Aceite todos os itens, após o termino da instalação feche o terminal/prompt e reabra, feito isso execute o seguinte comando:

conda install -c <https://conda.binstar.org/menpo> opencv3

4. - Após o termino da instalação:

4.1 – Ambiente Linux:

4.1.1 - Navegue através do terminal até o local onde foi clonado/descompactado os scripts do Cético ou digite o seguinte comando `python cetico.py`

4.2 – Ambiente Windows:

4.2.1 - Navegue até o local onde foi clonado/descompactado os scripts do Cético e execute o arquivo `cetico.bat` ou digite o seguinte comando `python <local onde encontra-se os scripts> cetico.py`

ANEXO A – _ILLUMINANTS.PY

```

import shutil
import platform
import os

from thirdparty.illuminants.sourcecode.extractGGEMaps import extractNewGrayWorldMaps
from thirdparty.illuminants.sourcecode.extractIICMaps import extractIlluminantMaps
from thirdparty.illuminants.sourcecode.extractAllFeatureVectors import main as
extractAllFeatureVectors
from thirdparty.illuminants.sourcecode.classifySVMCetico import fullClassification

programa = "Cético"
Modulo = "Illuminants"

def SegmentacaoDeimagens(path):
    database_file = os.path.dirname(__file__)+"/illuminants/data-base/images/"
    segmented_dir = os.path.dirname(__file__) + "/illuminants/data-base/segmented/"
    #os.system("rm -f " + segmented_dir + "*")
    copiarImagemDatabase(path)
    application_dir = os.path.dirname(__file__)+"/illuminants/illuminants/build/bin/"
    string = "python "+os.path.dirname(__file__) +
"/illuminants/sourcecode/segmentAllImagesForIlluminantMethod.py "+database_file+
"+segmented_dir+" "+application_dir
    os.system(string)
def copiarImagemDatabase(path):
    database_dir = os.path.dirname(__file__)
    database_dir += "/illuminants/data-base/images/"
    os.system("rm -f "+database_dir+"*")
    shutil.copy2(path, database_dir)
def extrairIIC():
    extractIlluminantMaps(os.path.dirname(__file__),"segmented","IIC")
def extrairGGE():
    extractNewGrayWorldMaps(os.path.dirname(__file__), "segmented", "GGE", 1, 1, 3)
def gerarTXTcomFacePositions(listadefaces):
    print("gerando arquivo facepositions:\n")
    sourcefolder = os.path.dirname(__file__)
    filedir = sourcefolder + "/illuminants/face-positions/"
    command = "rm -f " + filedir+ "*"
    os.system(command)
    im = os.listdir(sourcefolder+"/illuminants/data-base/segmented/")
def convertervaloreslista():
    posFaces = ""
    ID = 0
    for vetor in listadefaces:
        ID += 1
        # ordem dos vetores armazenados([x0, y0, w1, h1])
        xtopleft = int(vetor[0])
        ytopleft = int(vetor[1])
        xbottomright = int(vetor[2])+int(vetor[0])
        ybottomright = int(vetor[3])+int(vetor[1])

```

```

        posFaces += (str(ID)+"\tLABEL"+str(ID)+"\t"+str(xtopleft)+"\t"+str(xbottomright)
+ "\t"+str(ytopleft)+"\t"+str(ybottomright))+"\n"
    return posFaces
for i in im:
    tt = i.split(".")
    nome_arquivo = tt[0] + ".txt"
    arquivo = open(filedir+nome_arquivo, "w")
    print(nome_arquivo)
    type(arquivo)
    conteudoArquivo = convertervaloreslista()
    print(conteudoArquivo)
    arquivo.write(conteudoArquivo)
    arquivo.close()
return 0
def extrairFeaturesVector(descritores):
    folder = os.path.dirname(__file__)
    extractAllFeatureVectors(descritores, folder)
def limparPastasTemporarias():
    sourcefolder = os.path.dirname(__file__)
    # limpar pasta temp faces
    filedir = sourcefolder + "/temp/faces/"
    command = "rm -f " + filedir + "*"
    os.system(command)
    # limpar pasta temp vectors
    filedir = sourcefolder + "/illuminants/temp/vectors/"
    command = "rm -f " + filedir + "*"
    os.system(command)
def limparTudo():
    # limpa pasta face-positions"
    sourcefolder = os.path.dirname(__file__)
    filedir = sourcefolder + "/illuminants/face-positions/"
    command = "rm -f " + filedir+ "*"
    # limpa pasta data-base/images/
    os.system(command)
    filedir = sourcefolder + "/illuminants/data-base/images/"
    command = "rm -f " + filedir+ "*"
    os.system(command)
    # limpa pasta data-base/GGE/
    os.system(command)
    filedir = sourcefolder + "/illuminants/data-base/GGE/"
    command = "rm -f " + filedir+ "*"
    os.system(command)
    # limpa pasta data-base/IIC/
    os.system(command)
    filedir = sourcefolder + "/illuminants/data-base/IIC/"
    command = "rm -f " + filedir+ "*"
    os.system(command)
    # limpa pasta data-base/segmented/
    os.system(command)
    filedir = sourcefolder + "/illuminants/data-base/segmented/"

```



```

command = "rm -f " + filedir + "*"
os.system(command)
# limpa pasta data-base/faces/
os.system(command)
# limpar pasta extracted-feature-vectors
filedir = sourcefolder + "/illuminants/extracted-feature-vectors/"
command = "rm -rf " + filedir + "*"
os.system(command)
limparPastasTemporarias()
def classificadorSVMCetico(descritores):
    baseDir = os.path.dirname(__file__)
    imgDir = baseDir + "/illuminants/data-base/images/"
    im = os.listdir(imgDir)
    for i in im:
        print(i)
        tt = i.split(".")
        imagem = tt[0] + ".png"
    resultadoSVM = fullClassification(baseDir, descritores, imagem)
    return resultadoSVM

```

ANEXO B – Função illuminants em funcoes.py

```

from thirdparty._illuminants import *

def Moduloilluminant(operacao, imagePath, vetorDeFaces):
    if platform.system() == 'Windows':
        funcaoIndisponivel(platform.system())
        print(platform.system())
    else:
        print("iniciando módulo illuminants")
        limparTudo()
        print("Pastas limpas")
        SegmentacaoDeimagens(imagePath)
        print("Imagem Segmentada")
        extrairGGE()
        print("Extraído GGE")
        extrairIIC()
        print("Extraído IIC")
        gerarTXTcomFacePositions(vetorDeFaces)
        print("Gerada posições de faces")
        extrairFeaturesVector(operacao)
        print("Vetores extraídos")
        limparPastasTemporarias()
        print("Limpendo pastas temporárias")
        resultado =classificadorSVMCetico(operacao)
        print("Concluida analise SVM")
        analiseIlluminantsterminada()
    return resultado

```

ANEXO C – Chamadas de aviso e erro do Illumants no script “__error.py”

```
def erro_Illuminants():
```

```
    try:
```

```
        messagebox.showerror("Illuminants!", "Não existem marcações na imagem.\n Marque os pontos a serem analisados antes de entrar na função")
```

```
    except:
```

```
        print ("ERRO!", "Há marcações suficientes na imagem. Marque ao menos dois pontos a serem analisados antes de entrar na função.")
```

```
def analiseIlluminantsterminada():
```

```
    try:
```

```
        messagebox.showwarning("Illuminants!",  
                                "Extração Concluída.")
```

```
    except:
```

```
        print("Aviso!",  
              "Extração concluída.")
```

ANEXO D – Inserção de item no menu e funções na interface gráfica.

Trecho de código para inserção de uma entrada no menu da aplicação.

```
toolsmenu.add_command(label="illuminants", command=self.illuminants)
```

A chamada acima executa a seguinte função de verificação e chamada :

```
def illuminants(self):
    if self.VarIllum == False:
        if self.path != "":
            if self.marcas != [] and len(self.marcas)>1:
                try:
                    self.janelaIlluminants()
                    self.VarIllum = True
                except NameError:
                    erroModuloGenerico(str(NameError))
            else:
                erro_Illuminants()
        else:
            erroImagemNaoCarregada()
```

A função acima, se estiver tudo corretamente configurado, por sua vez executa a seguinte função, responsável tanto pela chamada do método presente no script funcoes.py, quanto pela criação das janelas que permitem setar os parâmetros que serão passados a elas:

```
def janelaIlluminants(self):
    sub = self
    def extrairDescritoresdaImagem(_self, v1, v2, v3, v4, v5, v6, v7, v8, v9, v0):
        comando = []
        if v0:
            comando = [ "acc", "bic", "ccv", "eoac", "las", "lch", "sasi", "spytec", "unser" ]
        else:
            if v1:
                comando.append("acc")
            if v2:
                comando.append("bic")
            if v3:
                comando.append("ccv")
            if v4:
                comando.append("eoac")
            if v5:
                comando.append("las")
            if v6:
                comando.append("lch")
            if v7:
                comando.append("sasi")
            if v8:
                comando.append("spytec")
```

```

        if v9:
            comando.append("UNSER")
        resultado = Moduloilluminant(comando, self.path, self.marcas)
        janelaResultado(_self, resultado)
        _self.fecharJanelasSubordinadas("Illuminants")
    def janelaResultado(_self, resultado):
        outClassification, votesNormal, votesFake, finalClass = resultado
        normal = IntVar()
        normal.set(votesNormal)
        fake = IntVar()
        fake.set(votesFake)
        final = IntVar()
        final.set(finalClass)
        #saida = IntVar()
        #saida.set(outClassification)
        _self.j_Resultilluminants = Toplevel(self.interface)
        _self.j_Resultilluminants.title("Resultado")
        _self.j_Resultilluminants.wm_protocol("WM_DELETE_WINDOW", lambda:
        _self.fecharJanelasSubordinadas("resultadoIllu"))
        _self.j_Resultilluminants.resizable(width=FALSE, height=FALSE)
        Label(_self.j_Resultilluminants, text="Resultado:", anchor=CENTER)
        Label(_self.j_Resultilluminants, text="Normal :").grid(column=0, row=1)
        Label(_self.j_Resultilluminants, text="Modificada :").grid(column=0, row=2)
        Label(_self.j_Resultilluminants, text="Classificação Final:").grid(column=0, row=3)
        Label(_self.j_Resultilluminants, textvariable = normal).grid(column=1, row=1)
        Label(_self.j_Resultilluminants, textvariable = fake).grid(column=1, row=2)
        Label(_self.j_Resultilluminants, textvariable = final).grid(column=1, row=3)
        #Label(_self.j_Resultilluminants, textvariable = saida).grid(column=1, row=4)
        Button(_self.j_Resultilluminants, underline=0, text=u"Fechar", command=lambda:
        _self.fecharJanelasSubordinadas("resultadoIllu")).grid(row=5, column=1)
    def janelaModulosExtracao(_self):
        def executar():
            extrairDescritoresdaImagem(_self, var1.get(), var2.get(), var3.get(), var4.get(),
            var5.get(), var6.get(), var7.get(), var8.get(), var9.get(), var0.get())
        def verificaChkBox():
            if var0.get():
                var1.set(0), var2.set(0), var3.set(0), var4.set(0), var5.set(0), var6.set(0),
            var7.set(0), var8.set(
                0), var9.set(0)
            if var1.get() or var2.get() or var3.get() or var4.get() or var5.get() or var6.get() or
            var7.get() or var8.get() or var9.get():
                var0.set(0)
            if var1.get() and var2.get() and var3.get() and var4.get() and var5.get() and var6.get()
            and var7.get() and var8.get() and var9.get():
                var0.set(0)
        _self.j_illuminants = Toplevel(self.interface)
        _self.j_illuminants.title("Illuminants")
        _self.j_illuminants.resizable(width=FALSE, height=FALSE)
        _self.j_illuminants.wm_protocol("WM_DELETE_WINDOW", lambda :
        _self.fecharJanelasSubordinadas("Illuminants"))

```

```

Label(_self.j_illuminants, text="Selecione os descritores desejados:").grid(column=0,
row=0)
var1 = IntVar()
var2 = IntVar()
var3 = IntVar()
var4 = IntVar()
var5 = IntVar()
var6 = IntVar()
var7 = IntVar()
var8 = IntVar()
var9 = IntVar()
var0 = IntVar()
Checkbutton(_self.j_illuminants, anchor=W, text="ACC", variable=var1, offvalue=0,
width=20, command=verificaChkBox).grid(column=0, row=1)
Checkbutton(_self.j_illuminants, anchor=W, text="BIC", variable=var2, offvalue=0,
width=20, command=verificaChkBox).grid(column=0, row=2)
Checkbutton(_self.j_illuminants, anchor=W, text="CCV", variable=var3, offvalue=0,
width=20, command=verificaChkBox).grid(column=0, row=3)
Checkbutton(_self.j_illuminants, anchor=W, text="EOAC", variable=var4, offvalue=0,
width=20, command=verificaChkBox).grid(column=0, row=4)
Checkbutton(_self.j_illuminants, anchor=W, text="LAS", variable=var5, offvalue=0,
width=20, command=verificaChkBox).grid(column=0, row=5)
Checkbutton(_self.j_illuminants, anchor=W, text="LCH", variable=var6, offvalue=0,
width=20, command=verificaChkBox).grid(column=0, row=6)
Checkbutton(_self.j_illuminants, anchor=W, text="SASI", variable=var7, offvalue=0,
width=20, command=verificaChkBox).grid(column=0, row=7)
Checkbutton(_self.j_illuminants, anchor=W, text="SPYTEC", variable=var8,
offvalue=0,
width=20, command=verificaChkBox).grid(column=0, row=8)
Checkbutton(_self.j_illuminants, anchor=W, text="UNSER", variable=var9, offvalue=0,
onvalue=1, width=20, command=verificaChkBox).grid(column=0, row=9)
Checkbutton(_self.j_illuminants, anchor=W, text="TODOS", variable=var0, offvalue=0,
width=20, command=verificaChkBox).grid(column=0, row=10)
Button(_self.j_illuminants, underline= 0, text=u"Cancelar", command=lambda:
_self.fecharJanelasSubordinadas("Illuminants")).grid(row=10, column=1)
Button(_self.j_illuminants, underline= 2, text=u"Executar",
command=executar).grid(row=10, column=2)
janelaModulosExtracao(self)

```